APPENDIX B

Lean 4 for Rust Programmers: Cheat Sheet

This appendix is a quick reference for reading and writing Lean when your instincts are Rust's. Chapter 2 explains everything here in detail; the tables are for looking things up. The longer examples come from `code/lean/Book/AppB/CheatSheet.lean`, which compiles with the book's toolchain.

In this appendix you will learn:

- the Lean spelling of the Rust constructs you use every day;
- where the two languages differ in ways that matter (totality, immutability, implicit arguments, propositions);
- the commands, notations and tactics that appear most often in the book.

B.1 Syntax side by side

Rust	Lean	Notes
`fn f(x: u64) -> u64 { ... }`	`def f (x : ℕ) : ℕ := ...`	no `return` needed
`fn g<T>(x: T) -> T`	`def g {α : Type} (x : α) : α`	`{}` = implicit, inferred
`|x| x + 1`	`fun x => x + 1` or `(· + 1)`	`·` is the argument
`f(a, b)`	`f a b`	application is juxtaposition
`let x = e;`	`let x := e`	
`if c { a } else { b }`	`if c then a else b`	
`match e { P => a, ... }`	`match e with | P => a | ...`	must be exhaustive
`struct S { a: A }`	`structure S where a : A`	
`S { a: 1 }`, `S { a: 2, ..s }`	`{ a := 1 }`, `{ s with a := 2 }`	also `(1)`
`enum E { A, B(u8) }`	`inductive E where | a | b (n : UInt8)`	constructors `E.a`, `.a`
`impl S { fn m(&self) }`	`def S.m (s : S) ...`	called as `s.m`
`trait T { fn f(&self); }`	`class T (α : Type) where f : α → ...`	
`impl T for S { ... }`	`instance : T S where f := ...`	
`fn h<X: T>(x: X)`	`def h {X : Type} [T X] (x : X)`	`[]` = instance argument
`#[derive(Debug, PartialEq)]`	`deriving Repr, BEq`	also `DecidableEq`, `Hashable`
`mod m { ... }` / `use m::*;`	`namespace m ... end m` / `open m`	
`Option<T>`: `Some(x)`, `None`	`Option α`: `some x`, `none`	
`Result<T, E>`: `Ok`, `Err`	`Except ε α`: `.ok`, `.error`	note the order of arguments
`e?`	`← e` inside `do`	
`Vec<T>`, `vec![1, 2]`	`Array α`, `#[1, 2]`	`List α`, `[1, 2]` in proofs
`HashMap<K, V>`	`Std.HashMap K V`	`import Std.Data.HashMap`
`String`, `format!("{x}")`	`String`, `s!"{x}"`	
`u64`, `i64`, `usize`	`UInt64`, `Int64`, `USize`	
(no equivalent)	`ℕ`, `ℤ`	unbounded naturals and integers
`bool`, `true`	`Bool`, `true`	also `Prop`: see below
`()`	`()` of type `Unit`	
`!`, `enum Void {}`	`Empty` (data), `False` (proposition)	
`(A, B)`, `p.0`	`A × B`, `p.1` / `p.fst`	
`fn main() { println!(...) }`	`def main : IO Unit := IO.println ...`	
`// comment`, `/* */`, `///`	`-- comment`, `/- -/`, `/-- -/`	

B.2 Functions and definitions

```
-- fn square(x: u64) -> u64 { x * x }
def square (x : ℕ) : ℕ := x * x

-- Generic, with the type parameter implicit (inferred at call sites):
-- fn twice<T>(f: impl Fn(T) -> T, x: T) -> T { f(f(x)) }
```

```
def twice {α : Type} (f : α → α) (x : α) : α := f (f x)

-- A closure: |n| n + 1
#guard twice (fun n => n + 1) 5 = 7

-- Pattern matching in the definition itself (like a `match` on the argument):
def fib : ℕ → ℕ
  | 0 => 0
  | 1 => 1
  | n + 2 => fib n + fib (n + 1)
```

Every definition must be **total**: it must return a value for every input, and recursion must terminate. Lean checks structural recursion automatically (each call on a smaller piece of the input), accepts other recursion when you prove termination with `termination_by` / `decreasing_by`, and accepts possibly non-terminating functions only when marked `partial`, which hides their bodies from proofs. `#guard` checks a `Bool` condition at compile time, like a compile-time `assert!`.

B.3 Structures and inductive types

```
-- struct Point { x: f64, y: f64 }  +  #[derive(Debug, Clone, PartialEq)]
structure Point where
  x : Float
  y : Float
  deriving Repr, BEq

-- impl Point { fn norm2(&self) -> f64 { ... } }   (methods live in the namespace)
def Point.norm2 (p : Point) : Float := p.x * p.x + p.y * p.y

def origin : Point := { x := 0, y := 0 }
def p : Point := ⟨3, 4⟩            -- anonymous constructor
def q : Point := { p with y := 0 } -- struct update syntax: Point { y: 0.0, ..p }

/-- info: 25.000000 -/
#guard_msgs in
#eval p.norm2                      -- dot notation: Point.norm2 p
```

```
-- enum Shape { Circle(f64), Rect { w: f64, h: f64 } }
inductive Shape where
  | circle (r : Float)
  | rect (w h : Float)
  deriving Repr

def Shape.area : Shape → Float
  | .circle r => 3.14159 * r * r
  | .rect w h => w * h

-- A recursive enum needs no Box: enum Tree { Leaf, Node(Box<Tree>, u64, Box<Tree>) }
inductive Tree where
  | leaf
  | node (left : Tree) (value : ℕ) (right : Tree)

def Tree.sum : Tree → ℕ
  | .leaf => 0
  | .node l v r => l.sum + v + r.sum
```

Values are immutable. "Updating" a structure builds a new one; Lean's runtime reuses the old memory in place when nothing else refers to it (Chapter 24). Recursive types need no `Box`: values of inductive types are heap-allocated and reference-counted behind the scenes. `.circle` is short for `Shape.circle` when the expected type is known, like Rust's `Self::Circle` in a `match`.

B.4 Option, Except and `do`

```
-- Option<T> and Result<T, E>
def safeDiv (a b : ℕ) : Option ℕ :=
  if b = 0 then none else some (a / b)

def parsePort (s : String) : Except String ℕ :=
  match s.toNat? with
  | some n => if n < 65536 then .ok n else .error s!"{n} is not a port"
  | none => .error s!"not a number: {s}"

/-- info: Except.ok 8080 -/
#guard_msgs in
#eval parsePort "8080"

-- The `?` operator is `←` inside `do`:
def addPorts (a b : String) : Except String ℕ := do
  let x ← parsePort a
  let y ← parsePort b
  return x + y
```

`do` notation works for every monad (Chapter 20): `Option`, `Except`, `IO`, `StateM` and your own. Inside `do`, `let x ← e` runs `e` and binds its result, returning early on failure, which is what Rust's `?` does for `Option` and `Result`.

B.5 Collections

```
-- Vec<u64> ~ Array ℕ (a contiguous array, updated in place when unshared)
def squares : Array ℕ := (Array.range 5).map square

/-- info: #[0, 1, 4, 9, 16] -/
#guard_msgs in
#eval squares

-- Linked lists are the default sequence in proofs: [1, 2, 3] : List ℕ
/-- info: [2, 4, 6] -/
#guard_msgs in
#eval [1, 2, 3].map (· * 2)

/-- info: 6 -/
#guard_msgs in
#eval [1, 2, 3].foldl (· + ·) 0

-- HashMap<String, u64>
def stock : Std.HashMap String ℕ :=
  (∅ : Std.HashMap String ℕ) |>.insert "bolts" 40 |>.insert "nuts" 75

/-- info: some 75 -/
#guard_msgs in
#eval stock.get? "nuts"
```

`Array` is the efficient, contiguous sequence for programs; `List` is the linked list that proofs prefer, because its inductive structure makes induction easy. `(· * 2)` is a function: each `·` stands for the next argument.

B.6 Type classes

```
-- trait Describe { fn describe(&self) -> String; }
class Describe (α : Type) where
```

```
  describe : α → String

-- impl Describe for Point { ... }
instance : Describe Point where
  describe p := s!"({p.x}, {p.y})"

-- fn show_all<T: Describe>(xs: &[T]) -> Vec<String>
def showAll {α : Type} [Describe α] (xs : List α) : List String :=
  xs.map Describe.describe
```

A class is a structure of operations; an instance is a value of that structure, found automatically by **instance resolution** (Chapter 14). Unlike Rust, Lean has no orphan rule and allows overlapping instances, resolved by priority. Classes can also contain *proofs*, so a Lean `Monoid` instance must prove associativity, which a Rust `impl` can only promise in a comment.

B.7 Imperative code

```
-- Imperative-looking code in `do` blocks: `let mut`, `for`, early `return`.
def firstNegativeIndex (xs : Array ℤ) : Option ℕ := Id.run do
  for h : i in [0:xs.size] do
    if xs[i] < 0 then return some i
  return none

def sumTo (n : ℕ) : ℕ := Id.run do
  let mut total := 0
  for i in [0:n+1] do
    total := total + i
  return total

/-- info: 5050 -/
#guard_msgs in
#eval sumTo 100

-- fn main() { println!("..."); }
def main : IO Unit := do
  let name := "Lean"
  IO.println s!"Hello from {name}"
```

`let mut`, `for`, `while` and early `return` are allowed inside `do` blocks, and are translated into pure functional code. `Id.run do ...` uses the "identity" monad to write imperative-looking code with no effects at all. In `for h : i in [0:xs.size]`, the name `h` is a proof that `i` is in range, which lets `xs[i]` be checked at compile time (Chapter 1 explains indexing with proofs).

B.8 Proofs

```
-- A theorem is a definition whose type is a proposition.
theorem square_eq (n : ℕ) : square n = n * n := rfl

example (a b : ℕ) : (a + b) ^ 2 = a ^ 2 + 2 * a * b + b ^ 2 := by ring

example (x y : ℤ) (h₁ : x < y) (h₂ : 0 < x) : 0 < x + y := by linarith

theorem fib_pos (n : ℕ) (h : 0 < n) : 0 < fib n := by
  induction n using fib.induct with
  | case1 => omega
  | case2 => decide
  | case3 n ih₁ ih₂ =>
```

```
    simp only [fib]
    omega
```

Lean	Meaning
`theorem name (args) : P := proof`	a named proof of the proposition `P`
`example : P := proof`	an anonymous proof (checked, then forgotten)
`by tac₁; tac₂` or one tactic per line	build the proof with tactics
`p ∧ q`, `p ∨ q`, `¬p`, `p → q`, `p ↔ q`	and, or, not, implies, iff
`∀ x, P x`, `∃ x, P x`	for all, there exists
`a = b`, `a ≠ b`, `a ≤ b`	equality, inequality, order
`⟨hp, hq⟩`	build an `∧`, an `∃` or a structure from its parts
`h.1`, `h.2`, `h.left`	take an `∧` apart
`Prop` vs `Bool`	propositions (proved) vs booleans (computed); `decide` converts decidable ones
`sorry`	an unproved hole; makes Lean warn; never used in this book

Appendix D describes every tactic the book uses.

B.9 Commands

Command	Does
`#eval e`	evaluate `e` and print the result
`#check e`	print the type of `e`
`#print name`	print a definition or a proof term
`#print axioms name`	list the axioms a theorem depends on
`#reduce e`	normalize `e` by unfolding definitions (slow; for small terms)
`#guard b`	fail unless the boolean `b` evaluates to `true`
`#guard_msgs in cmd`	check `cmd`'s output against the doc comment above
`set_option pp.all true in cmd`	print terms with every implicit detail

B.10 Things that surprise Rust programmers

- **Application binds tightest.** `f x + 1` is `(f x) + 1`; write `f (x + 1)` when you mean it.
- **Natural-number subtraction truncates.** In `ℕ`, `3 - 5 = 0`. Rust would panic (in debug builds); Lean returns `0` because the function must be total. Use `ℤ` when you need negative numbers.
- **Division by zero is defined.** In `ℕ` and `ℤ`, `x / 0 = 0`, again for totality. Theorems about division therefore usually carry a hypothesis such as `b ≠ 0`.

- **No implicit copying or borrowing.** Every value can be used any number of times; there is no ownership system in the types. (There is one in the runtime, through reference counting: Chapter 24.)
- **Types are values.** `Type` is itself a type, functions can take and return types, and a type can depend on a value (`Vector ℕ n`).
- **Indentation matters** inside `do` blocks, tactic blocks and `match` alternatives, as in Python.
- **Errors appear where they happen.** The editor reports each error at its position and continues; a proof with an error elsewhere still shows its goals.